SWINBURNE UNIVERSITY OF TECHNOLOGY

DEPLOYMENT PORTFOLIO TASK 1

Student: Sidhaarth Krishnan
Student ID: 104089870
Unit: SWE40006 Software Deployment and Evolution
Teaching period: Semester 2, 2026
Submission date: 17 August 2026
Declared target: Task 1.4 - High Distinction
Source repository: https://github.com/RealSid08/SWE40006_Task1
Distribution links: GitHub Release | WinGet PR #418824

Submission statement: I completed Tasks 1.1 through 1.4 on Windows. This report covers the applications, WiX authoring, validated MSI builds, installation and execution evidence, the required final Hello World uninstall, the acquired runtime dependencies used for Task 1.3, and the public WinGet submission for Task 1.4.

1. Executive summary

This report documents an end-to-end Windows deployment and distribution workflow using WiX Toolset 4.0.6, .NET 10, GitHub Releases, and WinGet. I built and packaged the Hello World and HashGuard applications for Tasks 1.1 and 1.2, added the acquired Newtonsoft.Json, CsvHelper, and Humanizer dependencies for Task 1.3, and published the final MSI with a WinGet manifest submission for Task 1.4. Hello World was installed, executed, and then uninstalled as required.

Outcome: Final verification produced 4 passing tests and both MSI projects built with 0 warnings and 0 errors. The GitHub Release asset was published with a verified SHA-256 digest, the WinGet manifest passed local validation, and pull request #418824 was submitted to microsoft/winget-pkgs.

1.1 Setup environment and toolchain

Host environment	Native Windows 11 Home, 64-bit, build 26200
VM details	Not applicable - the assignment was completed directly on a Windows PC rather than a macOS virtual machine
Solution	SWE40006.Task1.sln containing five application, test, and installer projects
Build target	Release configuration; win-x64 application publish and x64 MSI packages

Component	Installed version	Role
Visual Studio	Community 2026 18.9.0	.NET desktop workload and solution/project authoring
.NET	SDK 10.0.400; Desktop Runtime 10.0.11	Build, test, publish, and execute the applications
WiX Toolset	CLI and SDK 4.0.6	Compile the two x64 MSI packages
HeatWave	Visual Studio extension installed	WiX project templates and Visual Studio integration

1.2 Visual Studio project evidence

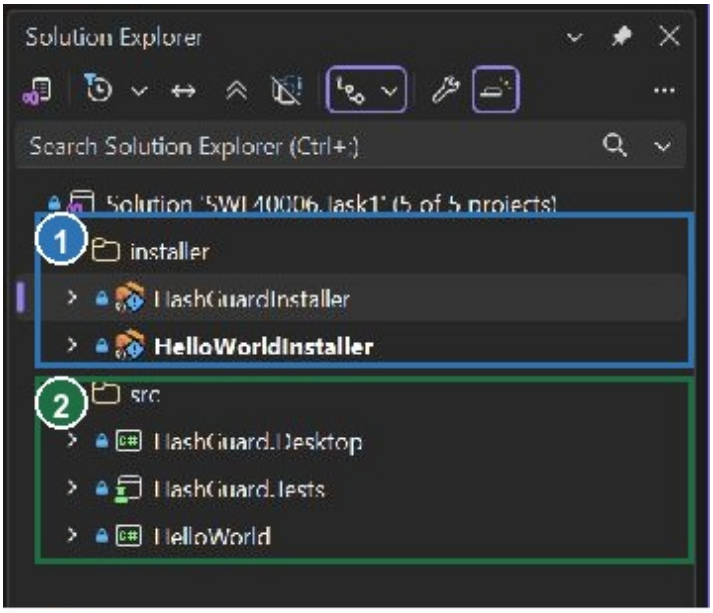

Figure 1. Annotated Visual Studio Solution Explorer showing the two WiX installer projects and the three application/test projects.

1.3 Step-by-step setup and build workflow

1. **Set up the Windows toolchain:** I installed and used Visual Studio Community 2026 18.9.0 with the .NET desktop development tools, .NET SDK 10.0.400, .NET Desktop Runtime 10.0.11, WiX Toolset 4.0.6, and the HeatWave Visual Studio extension.
2. **Open and verify the solution:** I opened SWE40006.Task1.sln in Visual Studio and confirmed that HelloWorld, HashGuard.Desktop, HashGuard.Tests, HelloWorldInstaller, and HashGuardInstaller all loaded correctly.

3. **Restore, build, and test the applications:** I restored the pinned NuGet packages, built the C# projects in Release mode, and ran HashGuard.Tests. The final run completed with four passed tests and no failures or skipped tests.
4. **Publish the deployable application files:** I published Hello World and HashGuard for win-x64 as framework-dependent applications. Each Release payload contained the executable, application DLL, dependency manifest, and runtime configuration required by its installer.
5. **Author the WiX packages:** I wrote the .wxs files to define the installation directories, application files, product identity, upgrade behaviour, and shortcuts. For Task 1.3, I added separate WiX components for Newtonsoft.Json.dll, CsvHelper.dll, and Humanizer.dll.
6. **Build both MSI packages:** I compiled HelloWorldInstaller and HashGuardInstaller in Release mode. Both builds produced the expected x64 MSI with 0 warnings and 0 errors.
7. **Install and inspect the deployed files:** I installed each MSI with verbose Windows Installer logging, then checked the exit status and installed directories. Both logs ended with status 0, and the expected files were present.
8. **Execute and verify the installed applications:** I ran Hello World from its installed directory and captured its banner before uninstalling the package. I then launched HashGuard, calculated both checksums for the sample payload, and confirmed that a history record was saved. Final verification showed that the Hello World product registration and application directory had been removed.

1.4 Console proof of successful builds and installs

Final Release build output - evidence/31 and evidence/32

```
HelloWorldInstaller -> ...\SWE40006-HelloWorld-1.0.0-x64.msi
Build succeeded.
    0 Warning(s)
    0 Error(s)

HashGuardInstaller -> ...\HashGuard-Desktop-1.0.0-x64.msi
Build succeeded.
    0 Warning(s)
    0 Error(s)
```

Final Windows Installer log excerpts - evidence/43 and evidence/44

```
Windows Installer installed the product. Product Name: SWE40006 Hello World.
Installation success or error status: 0.
MainEngineThread is returning 0

Windows Installer installed the product. Product Name: HashGuard Desktop.
Installation success or error status: 0.
MainEngineThread is returning 0
```

1.5 Reproducibility controls

I pinned the WiX SDK and extension versions in the .wixproj files and locked the NuGet dependency versions. Stable product and upgrade GUIDs keep the package identity consistent. The final test, build, installation, execution, and metadata logs are retained in the evidence directory, and both MSI SHA-256 hashes are recorded in evidence/33-msi-metadata.txt.

2. Task 1.1 - Pass: Hello World MSI

For Task 1.1, I built a small .NET console program that prints an assignment-specific banner. A WiX x64 package installs the published executable and its three companion runtime files to %LOCALAPPDATA% \Programs\SWE40006 Hello World. I installed the MSI silently, ran the deployed executable from that directory, and captured its output. I then uninstalled the package as required. Windows Installer returned status 0, and the final check confirmed that the product registration and application directory had been removed.

MSI	SWE40006-HelloWorld-1.0.0-x64.msi
Product version	1.0.0
Product code	{40C76FDB-606D-4DE6-9B17-F06149341B0A}
Install result	Exit code 0; four application files present
Execution result	Installed application printed the expected Task 1.1 banner
Final state	Package uninstalled; product registration and application directory removed

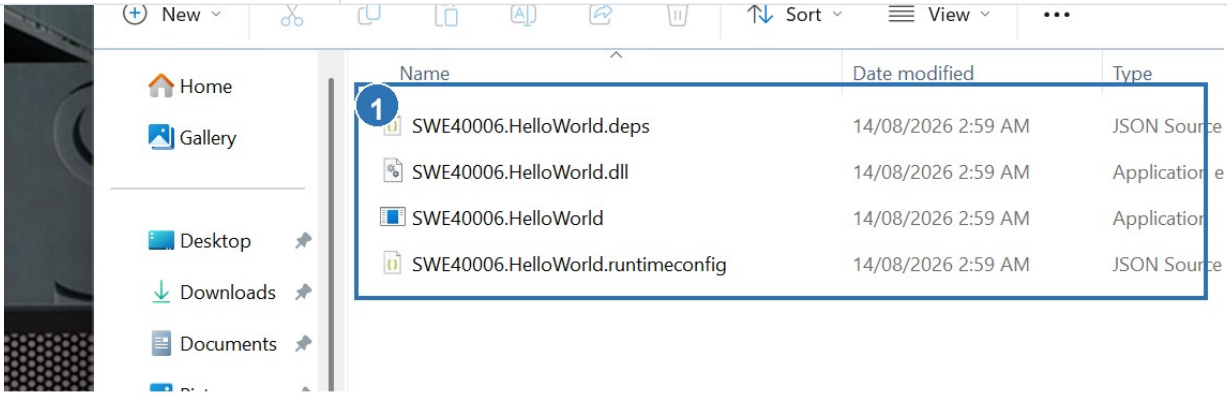

Figure 2. Annotated installed-directory evidence for the Task 1.1 Hello World package.

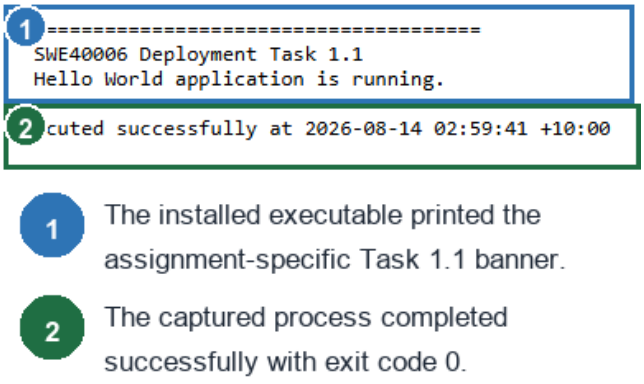

Figure 3. Annotated console output from the installed Hello World executable.

3. Task 1.2 - Credit: HashGuard Desktop

For Task 1.2, I developed HashGuard Desktop as an original C# WinForms application for calculating SHA-256 and SHA-512 file checksums. The application performs asynchronous hashing, can compare a result with an expected hash, stores a local JSON audit history, and exports history to CSV. The interface also identifies the external dependencies and displays complete hash values for copying and verification.

3.1 Application design

Layer	Implementation	Responsibility
Presentation	MainForm.cs	File selection, progress, hash display, comparison, history, and export actions
Hashing	HashService.cs	Buffered asynchronous SHA-256/SHA-512 computation using IncrementalHash
Persistence	HistoryStore.cs	Auditable local JSON history using Newtonsoft.Json
Export	CsvExportService.cs	CSV output through CsvHelper
Presentation helper	Humanizer	Readable dates and display text

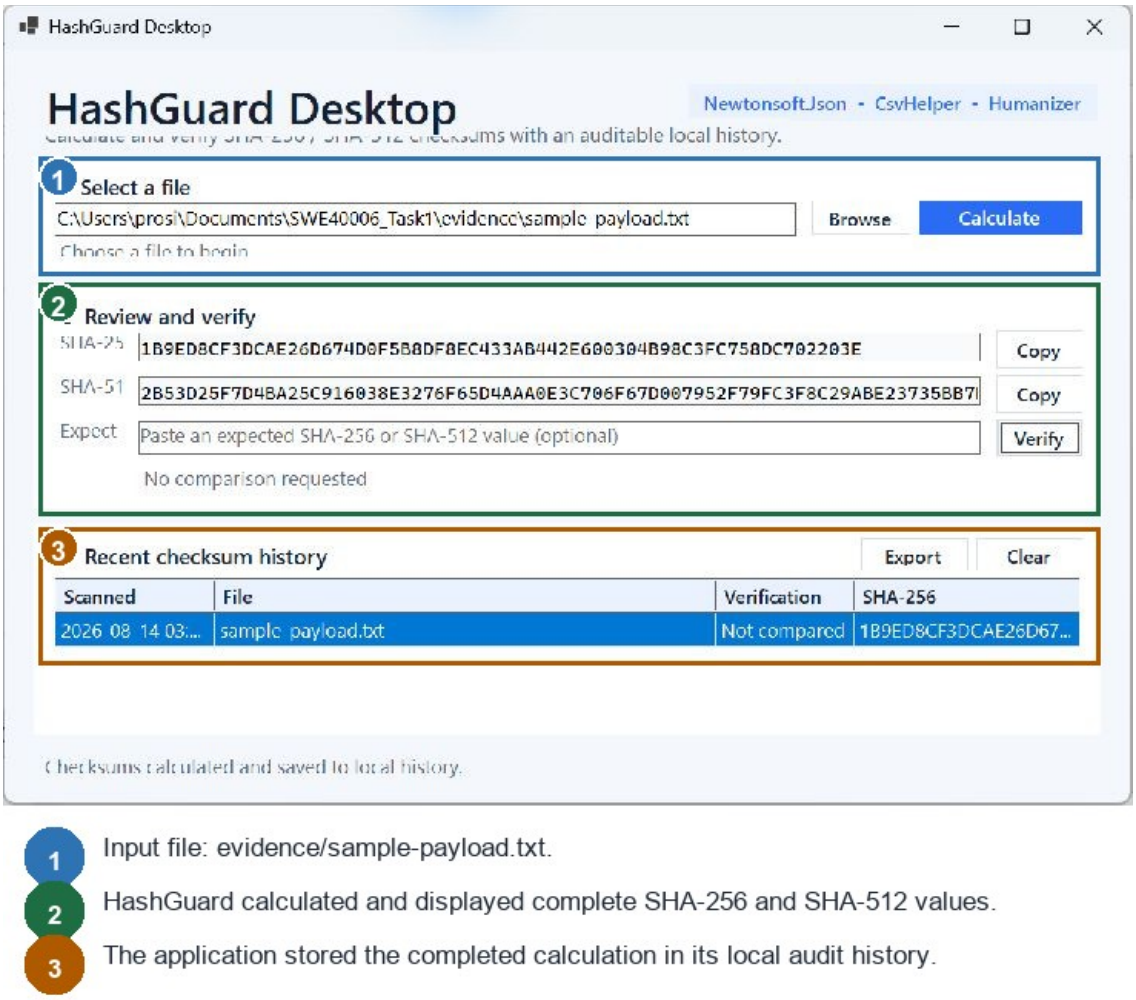

Figure 4. Annotated execution evidence from the installed HashGuard application.

3.2 Core checksum implementation

C# excerpt - HashService.ComputeAsync

```
        using var sha256 = IncrementalHash.CreateHash(HashAlgorithmName.SHA256);
        using var sha512 = IncrementalHash.CreateHash(HashAlgorithmName.SHA512);
        await using var stream = new FileStream(
            filePath,
            FileMode.Open,
            FileAccess.Read,
            FileShare.Read,
            BufferSize,
            FileOptions.Asynchronous | FileOptions.SequentialScan);

        var buffer = new byte[BufferSize];
        long totalRead = 0;
        int bytesRead;

        while ((bytesRead = await stream.ReadAsync(buffer, cancellationToken)) > 0)
        {
            sha256.AppendData(buffer, 0, bytesRead);
            sha512.AppendData(buffer, 0, bytesRead);
            totalRead += bytesRead;
            progress?.Report(fileInfo.Length == 0 ? 100 : (int)(totalRead * 100 / fileInfo.Length));
        }

        progress?.Report(100);
        return new HashResult(
            fileInfo.FullName,
            fileInfo.Length,
            Convert.ToHexString(sha256.GetHashAndReset()),
            Convert.ToHexString(sha512.GetHashAndReset()));
```

HashService streams each file in 1 MiB buffers and appends every buffer to both hash algorithms. This avoids loading the entire file into memory and allows the interface to display progress for large inputs.

3.3 Automated verification

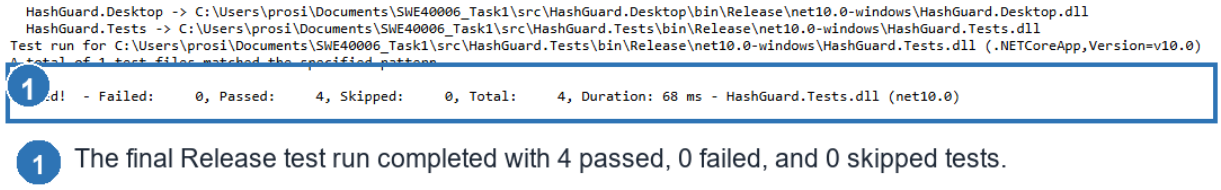

Figure 5. Annotated console output from the final automated test run.

Test	Expected behaviour	Result
Known hashes	SHA-256 and SHA-512 match known values	Passed
Mismatch verification	Malformed or different expected hash is rejected	Passed
JSON round trip	Newtonsoft.Json persists and restores a history record	Passed
CSV export	CsvHelper writes an export containing the history data	Passed

4. Task 1.3 - Distinction: external dependencies

For Task 1.3, I added each acquired runtime dependency as an individual WiX component. The final HashGuard installation was then verified to contain Newtonsoft.Json.dll, CsvHelper.dll, and Humanizer.dll alongside the application.

4.1 Runtime dependency set

Dependency	Pinned version	Use in HashGuard
Newtonsoft.Json	13.0.4	Serializes and deserializes checksum history
CsvHelper	33.1.0	Exports the audit history to CSV
Humanizer.Core	3.0.10	Formats readable time and display values

4.2 WiX authoring

WiX v4 excerpt - explicit external DLL components

```
<Component Id="NewtonsoftJsonComponent" Guid="{9AAAABB3-2199-4AFB-B3F9-14AAAB88A66A}">
  <File Id="NewtonsoftJsonDll" Source="..\..\src\HashGuard.Desktop\bin\Release\net10.0-windows\win-
x64\publish\Newtonsoft.Json.dll" />
  <RegistryValue Root="HKCU" Key="Software\SidharthKrishnan\HashGuardDesktop\Components"
Name="NewtonsoftJson" Type="integer" Value="1" KeyPath="yes" />
</Component>
<Component Id="CsvHelperComponent" Guid="{F6EBD07F-2291-4527-9317-4739766DD50B}">
  <File Id="CsvHelperDll" Source="..\..\src\HashGuard.Desktop\bin\Release\net10.0-windows\win-
x64\publish\CsvHelper.dll" />
  <RegistryValue Root="HKCU" Key="Software\SidharthKrishnan\HashGuardDesktop\Components"
Name="CsvHelper" Type="integer" Value="1" KeyPath="yes" />
</Component>
<Component Id="HumanizerComponent" Guid="{EA9E4267-5AA4-4E47-9AAA-463335F372FD}">
  <File Id="HumanizerDll" Source="..\..\src\HashGuard.Desktop\bin\Release\net10.0-windows\win-
x64\publish\Humanizer.dll" />
  <RegistryValue Root="HKCU" Key="Software\SidharthKrishnan\HashGuardDesktop\Components"
Name="Humanizer" Type="integer" Value="1" KeyPath="yes" />
</Component>
```

I assigned each acquired DLL its own stable component GUID and used an HKCU registry value as the per-user key path required by full MSI validation. The same package also defines the application executable, application DLL, dependency manifest, runtime configuration, installation directory, cleanup components, and Start Menu shortcut.

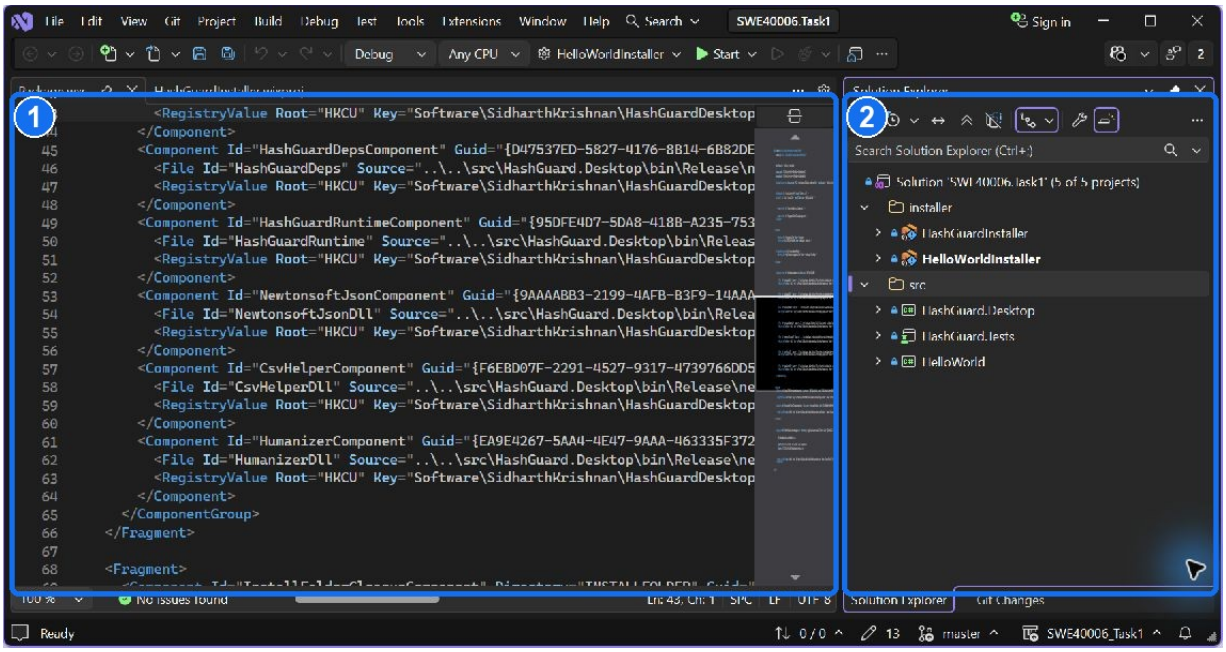

Figure 6. Final Visual Studio view of Package.wxs after the full-validation corrections.

Annotations: *1 shows the acquired DLL components with HKCU registry key paths; 2 confirms the complete five-project solution in Visual Studio.*

4.3 Build and installed-file evidence

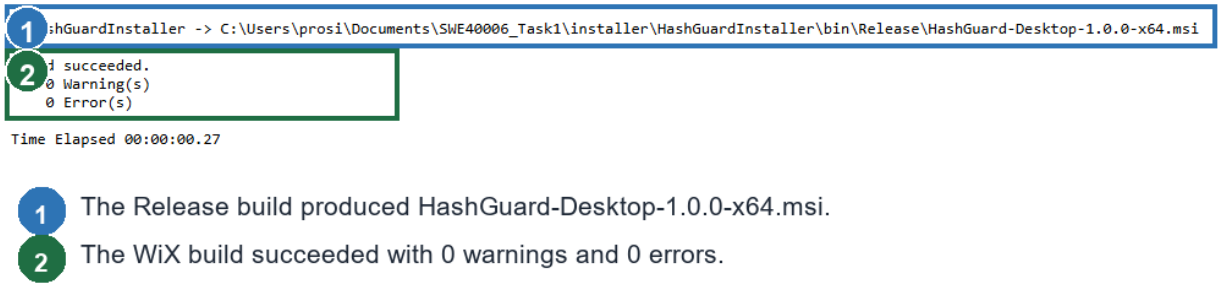

Figure 7. Annotated console output from the final HashGuard MSI build.

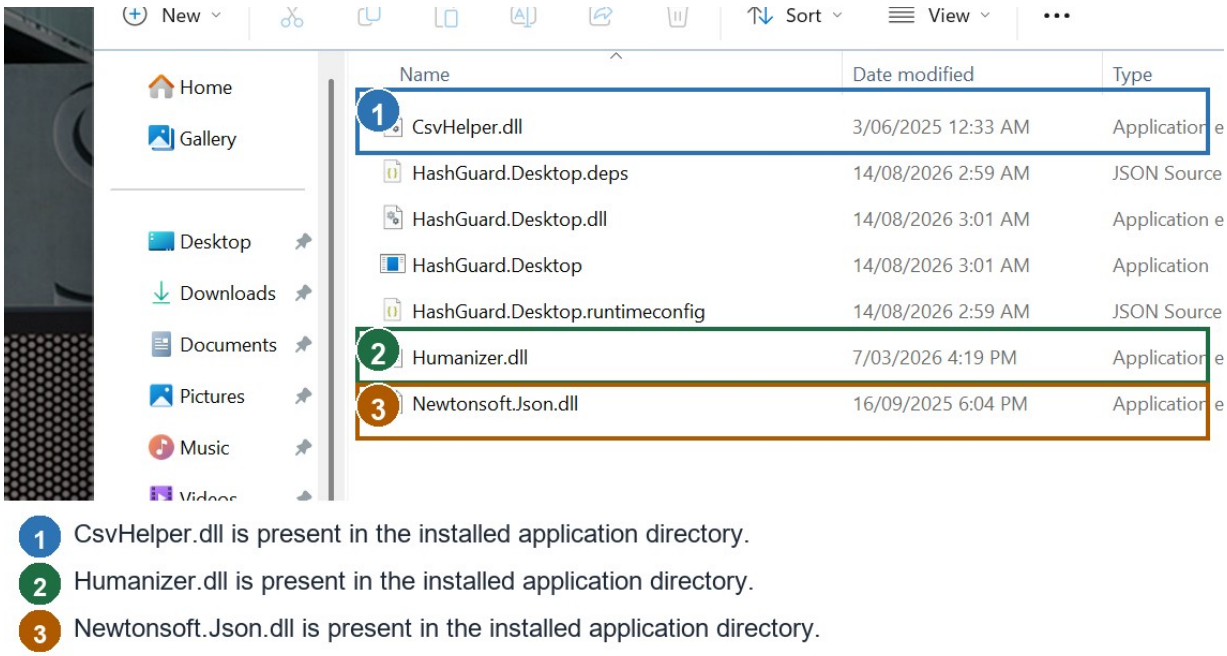

Figure 8. Annotated installed-directory evidence for the three external runtime DLLs.

4.4 MSI identity and installation

MSI	HashGuard-Desktop-1.0.0-x64.msi
Product code	{C57B16A8-E9E5-4EE9-8020-EE069EB409A7}
Upgrade code	{23AB1A31-A44A-41FE-BBDA-5245030BAD76}
Size	1,097,728 bytes
SHA-256	A947612491037D6512AFA6AECEC5838628D7A8B899F6520B476F42FE7AFE6A5B
Final install	Exit code 0; 7 files plus Start Menu shortcut
Execution	Installed application launched and processed evidence/sample-payload.txt

5. Task 1.4 - High Distinction: WinGet distribution

For Task 1.4, I published package identifier SidharthKrishnan.HashGuardDesktop at version 1.0.0. The public release, manifest URL, MSI digest, package identity, and official repository submission all refer to the same validated installer.

5.1 Release preparation and MSI validation

Before publishing the release, I reran the HashGuard tests, published the win-x64 application, and rebuilt the WiX installer with Windows Installer validation enabled. The final build completed with 0 warnings and 0 errors. Installing the release MSI returned exit code 0 and placed the seven expected files in the HashGuard Desktop directory.

Release	https://github.com/RealSid08/SWE40006_Task1/releases/tag/v1.0.0
MSI asset	HashGuard-Desktop-1.0.0-x64.msi
File size	1,097,728 bytes
SHA-256	A947612491037D6512AFA6AECEC5838628D7A8B899F6520B476F42FE7AFE6A5B
Product code	{C57B16A8-E9E5-4EE9-8020-EE069EB409A7}

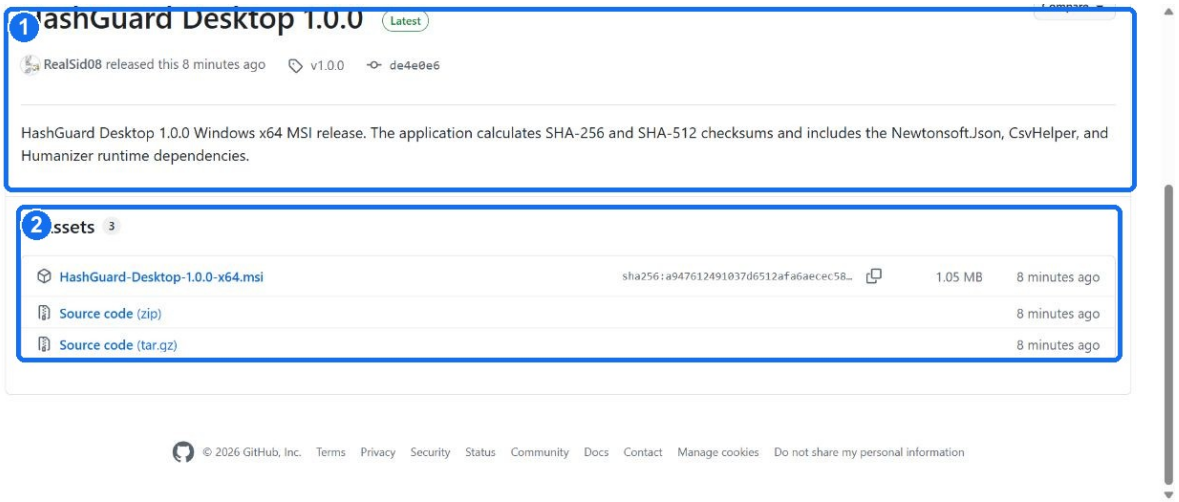

Figure 9. Public GitHub Release containing the final HashGuard x64 MSI asset.

Annotations: *1 identifies the public v1.0.0 release; 2 identifies the downloadable MSI asset and GitHub digest.*

5.2 WinGet manifest authoring and validation

I created the required three-file WinGet layout: a version manifest, an x64 installer manifest, and an en-US default-locale manifest. The installer manifest declares the WiX installer type, user scope, product code, public release URL, exact SHA-256 digest, and Microsoft.DotNet.DesktopRuntime.10 as a package dependency with minimum version 10.0.0.

```
PackageIdentifier: SidharthKrishnan.HashGuardDesktop
PackageVersion: 1.0.0
InstallerType: wix
Scope: user
Dependencies:
  PackageDependencies:
  - PackageIdentifier: Microsoft.DotNet.DesktopRuntime.10
    MinimumVersion: 10.0.0
InstallerUrl: https://github.com/RealSid08/SWE40006_Task1/releases/download/v1.0.0/HashGuard-Desktop-1.0.0-x64.msi
InstallerSha256: A947612491037D6512AFA6AECEC5838628D7A8B899F6520B476F42FE7AFE6A5B

> winget validate --manifest manifests\s\SidharthKrishnan\HashGuardDesktop\1.0.0
Manifest validation succeeded.
```

I checked the dependency identifier through WinGet, which resolved Microsoft.DotNet.DesktopRuntime.10 at version 10.0.11. I also downloaded the MSI from the release URL and hashed it again; the digest matched the manifest exactly.

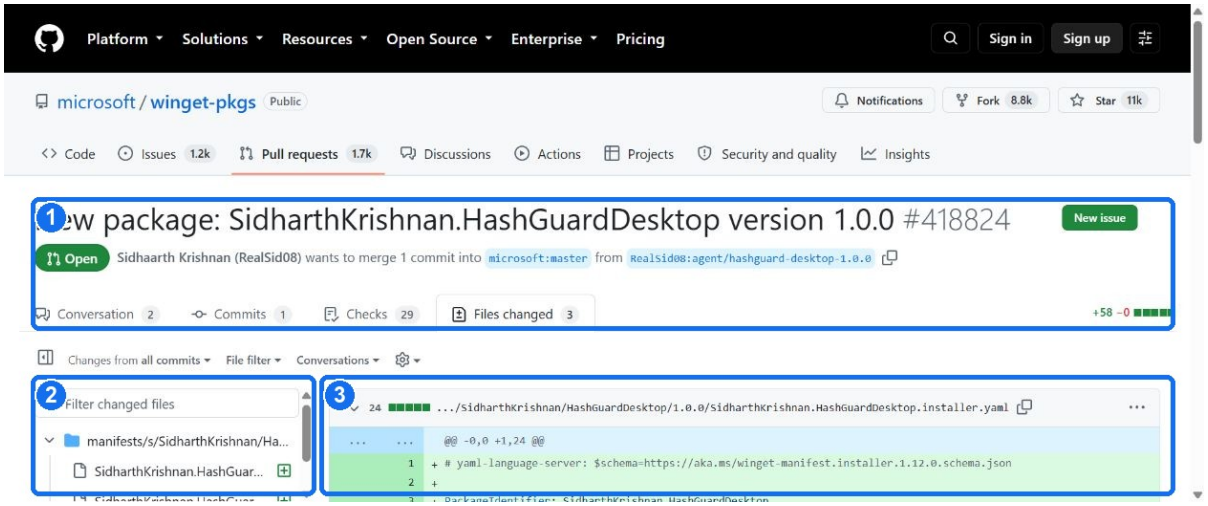

Figure 10. Three WinGet manifest files submitted to the official repository.

Annotations: *1 confirms the official repository and open PR; 2 shows the three-file manifest tree; 3 shows the submitted installer YAML diff.*

5.3 Official WinGet repository submission

The manifest was committed on branch `agent/hashguard-desktop-1.0.0` in the RealSid08/winget-pkgs fork and submitted to [microsoft/winget-pkgs pull request #418824]. The pull request is open against microsoft:master with one commit, three manifest files, and 58 added lines. The initial pull request, manifest, URL, domain, and manifest-policy checks all passed.

Pull request	https://github.com/microsoft/winget-pkgs/pull/418824
Repository	microsoft/winget-pkgs
State	Open; ready for repository review
Change	3 manifest files; 58 additions
Initial validation	PR, manifest, URL, domain, and policy checks passed

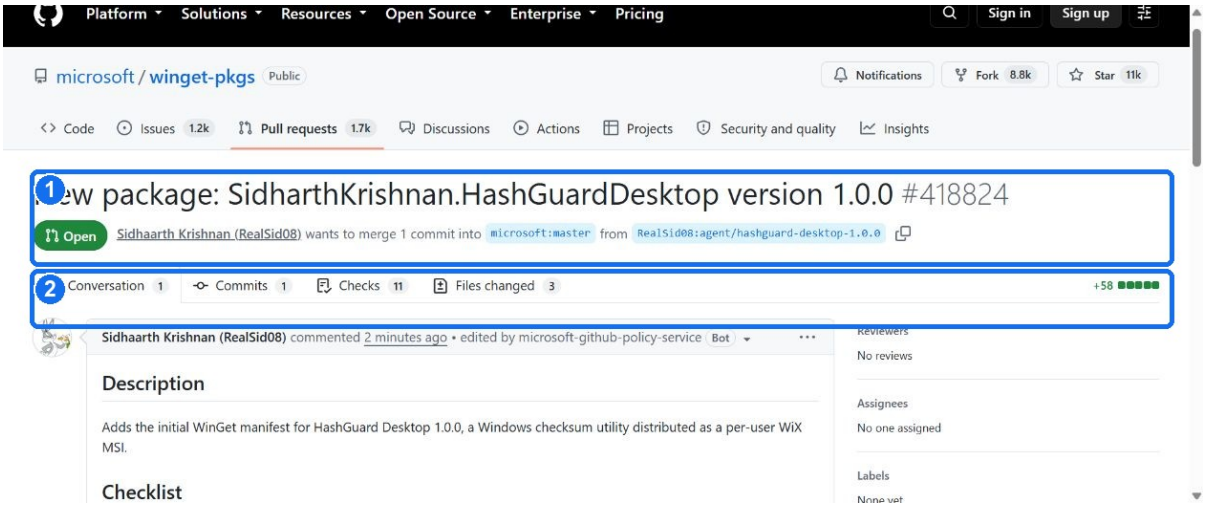

Figure 11. Open package manifest pull request in microsoft/winget-pkgs.

Annotations: *1 identifies the package, PR number, open state, and branch direction; 2 records the commit and three changed manifest files.*

6. Manual checksum result

Input	evidence/sample-payload.txt
SHA-256	1B9ED8CF3DCAE26D674D0F5B8DF8EC433AB442E600304B98C3FC758DC702203E
SHA-512	2B53D25F7D4BA25C916038E3276F65D4AAA0E3C706F67D007952F79FC3F8C29AB E23735BB7D6E1A41C3AE4A89D9EE1694A82E1894C4A9EBC3F1B121A640C47AF
History	One record written to the local JSON audit history

7. Troubleshooting and engineering decisions

7.1 WinForms designer serialization warning

During the initial compilation, I encountered WFO1000 because WinForms designer serialization interpreted public form state as design-time content. I refactored the form so its application services and state use private readonly fields initialized in the constructor. This separated runtime state from designer serialization and restored a clean build.

7.2 WiX v4 UI migration

My initial WiX build used the WiX v3 UIRef pattern and failed with WIX0094 because that identifier is protected in WiX v4. I migrated the package to the WiX v4 UI extension namespace using ui:WixUI with WixUI_InstallDir and INSTALLFOLDER. WixToolset.UI.wixext remains pinned to version 4.0.6.

7.3 Installed-app runtime failure

The first installed HashGuard build terminated before displaying its window. I traced the failure in the Windows Application Event Log to a .NET 10 NotSupportedException caused by a transparent button border setting. After I set BorderSize to 0, the tests, publish, MSI build, installation, and installed-application execution all completed successfully.

7.4 Full MSI validation

Full Windows Installer validation initially reported ICE38 and ICE64 because files under the per-user profile were being used as component key paths. I changed each file component to use an HKCU registry value as its key path and added cleanup components for the installation folders. I suppressed ICE91 because the package is explicitly per-user. The rebuilt MSI then completed full validation with 0 warnings and 0 errors.

8. Reproduction procedure

I ran the following commands from the repository root on Windows with .NET 10 and WiX Toolset 4.0.6 installed. Because the project dependencies are pinned, the same commands reproduce the test, publish, build, and installation results.

PowerShell - build and test

```
dotnet test src/HashGuard.Tests/HashGuard.Tests.csproj -c Release
dotnet publish src/HashGuard.Desktop/HashGuard.Desktop.csproj -c Release -r win-x64 --self-contained
false
dotnet build installer/HelloWorldInstaller/HelloWorldInstaller.wixproj -c Release
dotnet build installer/HashGuardInstaller/HashGuardInstaller.wixproj -c Release
```

PowerShell - install

```
msiexec /i installer\HelloWorldInstaller\bin\Release\SWE40006-HelloWorld-1.0.0-x64.msi /qn /L*v
evidence\43-restored-hello-install.log
msiexec /i installer\HashGuardInstaller\bin\Release\HashGuard-Desktop-1.0.0-x64.msi /qn /L*v
evidence\44-restored-hashguard-install.log
```

PowerShell - final Hello World uninstall

```
msiexec /x {40C76FDB-606D-4DE6-9B17-F06149341B0A} /qn /norestart /L*v evidence\48-final-hello-
uninstall.log
```

PowerShell - validate the WinGet manifest and verify the public asset

```
winget validate --manifest distribution\winget\1.0.0
Get-FileHash .\HashGuard-Desktop-1.0.0-x64.msi -Algorithm SHA256
# Expected SHA-256:
# A947612491037D6512AFA6AECEC5838628D7A8B899F6520B476F42FE7AFE6A5B
```

8.1 Expected outputs

Step	Success condition	Final observed result
Test	All automated tests pass	4/4 passed
Build	Both MSI projects succeed	0 warnings; 0 errors
Install	MSI exits 0 and files appear	Both exits 0; 4 and 7 files respectively
Run	Installed executable starts	Hello banner and HashGuard checksum result captured
Uninstall	Hello World MSI exits 0 and installed files are removed	Exit 0; product registration and application directory absent
Release	MSI is publicly downloadable and its digest is fixed	v1.0.0 asset published; 1,097,728 bytes; SHA-256 verified
Distribute	Valid manifest PR is submitted to the official repository	microsoft/winget-pkgs PR #418824 is open with three manifest files

9. References

FireGiant. HeatWave Community Edition and Visual Studio integration. https://docs.firegiant.com/heatwave/

FireGiant. WiX Toolset v4 UI extension (WixUI). https://docs.firegiant.com/wix/tools/wixext/wixui/

Microsoft. IncrementalHash class. https://learn.microsoft.com/dotnet/api/system.security.cryptography.incrementalhash

NuGet. Newtonsoft.Json 13.0.4. https://www.nuget.org/packages/Newtonsoft.Json/13.0.4

NuGet. CsvHelper 33.1.0. https://www.nuget.org/packages/CsvHelper/33.1.0

NuGet. Humanizer.Core 3.0.10. https://www.nuget.org/packages/Humanizer.Core/3.0.10

Swinburne University of Technology. Deployment Portfolio Task 1. https://swinburne.instructure.com/courses/77400/assignments/796862

Microsoft. Windows Package Manager Community Repository. https://github.com/microsoft/winget-pkgs

GitHub Release. HashGuard Desktop 1.0.0. https://github.com/RealSid08/SWE40006_Task1/releases/tag/v1.0.0

Microsoft WinGet pull request #418824. https://github.com/microsoft/winget-pkgs/pull/418824